Secure Core Banking System (SCBS)

Deliverable 3 - Final Security Report

Security Testing Analysis | Secure Code Review | Final Security Fixes | Final Demo

Course	Secure Software Development and Development (CY321)
Deliverable	Deliverable 3
Project Type	Secure Flask-based core banking web application
Repository Branch	frontend
Submission Date	02 May 2026
Team Members	Ali Ahmed Chaudhry 2023093, Muhammad Ismail 2023452, Shayan Siddiqui 2023656

Repository: https://github.com/Ismail-005/Secure-Software-Design-Project/tree/frontend

1. Executive Summary

Secure Core Banking System (SCBS) is a role-based banking dashboard implemented with Python, Flask, SQLite, SQLAlchemy, Flask-WTF, Flask-Login, Flask-Session, and Flask-Migrate. The application focuses on controlled money-movement workflows, account management, fraud review, audit visibility, and secure authentication. Deliverable 3 completes the security-focused phase of the semester project by documenting security testing analysis, secure testing techniques, secure code review, final security fixes, threat model coverage, security features, test results, and demo readiness.

The implementation contains layered boundaries: routes handle user-facing request validation, services enforce business and security rules, repositories isolate database access, and middleware centralizes access control, rate limiting, and session protection.

2. Deliverable 3 Scope Mapping

Deliverable 3 Requirement	How SCBS Satisfies It
Security Testing Analysis	Security tests cover brute-force lockout, rate limiting, unauthorized access rejection, admin-only audit visibility, replay attack rejection, SQL injection payload handling, and XSS escaping.
Secure Testing Techniques and Analysis	The test suite is divided into integration, security, and unit tests. Integration tests validate Flask routes through the test client; unit tests validate services, repositories, middleware, fraud logic, audit integrity, and transaction rules.
Secure Code Review	The review focuses on authentication, MFA/OTP, session handling, RBAC, CSRF, input validation, database access, financial transaction integrity, audit log integrity, and configuration hardening.
Final Security Fixes and Enhancements	The current implementation includes placeholder secret rejection, CSRF-enabled production configuration, session hardening, rate limit controls, replay protection, fraud detection, tamper-evident audit logs, and frontend escaping through Jinja templates.
Final Report	This document consolidates the threat model, security features, testing approach, test results, limitations, and final demo plan.
Source Code Submission	The source code is maintained in the GitHub repository branch named frontend.
Live Demo and Presentation	The report includes a demo path covering login, MFA, account dashboard, transactions, fraud review, audit logs, admin controls, and security rejection cases.

3. System Overview

SCBS simulates a secure internal banking platform. The core workflows are customer sign-up, staff and customer authentication, OTP verification for coursework MFA, account dashboard access, deposit, withdrawal, transfer, transaction history, fraud review, audit log review, and admin user management.

- Backend: Python 3.11 and Flask.
- Database layer: SQLite with SQLAlchemy and migrations through Flask-Migrate.
- Security extensions: Flask-WTF, Flask-Login, Flask-Session, bcrypt, pytest, and pytest-flask.
- Architecture: Routes -> Services -> Repositories -> SQLite.
- Primary roles: customer, teller, manager, and admin.

4. Final Threat Model

4.1 Protected Assets

- User credentials and password hashes.
- Authenticated sessions and OTP verification state.
- Customer identities, account data, and transaction history.
- Ledger entries used to calculate account balances.
- Fraud flags and review queues.
- Audit logs and audit hash-chain integrity values.
- Administrative privileges and role assignments.

4.2 Trust Boundaries

Boundary	Risk	Control
Browser -> Flask routes	Malicious form input, CSRF, XSS payloads, unauthenticated access.	Flask-WTF CSRF, route validation, login requirements, Jinja escaping, RBAC decorators/middleware.
Routes -> Services	Bypassing business rules or role checks.	Security-sensitive operations are centralized in service methods and role-aware middleware.
Services -> Repositories	SQL injection or inconsistent persistence.	Repository pattern and SQLAlchemy parameterized access.
Transaction request -> Ledger	Replay, insufficient funds, partial updates, incorrect balance.	Nonce-based replay checks, insufficient-funds checks, atomic transaction logic, ledger-backed balances.
Security events -> Audit logs	Silent tampering or missing accountability.	HMAC-backed hash-chained audit logs for tamper evidence.
Configuration -> Runtime app	Weak secrets, unsafe defaults, debug exposure.	Startup rejection for missing/placeholder SECRET_KEY, debug not forced on, secure cookie attributes.

4.3 STRIDE-Based Threat Analysis

STRIDE Category	Representative Threat	SCBS Mitigation
Spoofing	Attacker tries to log in as another user.	bcrypt password verification, OTP verification step, failed-login tracking, account lockout.
Tampering	Attacker modifies transaction or audit data.	Atomic transaction flow, ledger-backed balance calculation, HMAC hash-chain audit logs.
Repudiation	User denies transaction or admin action.	Audit logs record login, transaction, lockout, and security events.
Information Disclosure	Customer accesses another user account or admin-only pages.	Role-based access control and ownership checks.
Denial of Service	Repeated login or transaction attempts overload routes.	Rate limiting and lockout for abusive login behavior.
Elevation of Privilege	Customer attempts to access admin functions.	RBAC restrictions and route-level authorization checks.

5. Implemented Security Features

Control Area	Implementation Summary
Authentication	Username/password login with bcrypt password hashing and verification.
MFA / OTP	Coursework demo OTP verification flow after password authentication. OTP expiry is configurable.
Brute-force defense	Failed login tracking and account lockout after repeated failures.
Rate limiting	Configurable request throttling window and maximum attempt limits.
Session protection	Session timeout, IP-bound session guard, HTTP-only cookies, SameSite=Lax cookies.
CSRF protection	Flask-WTF CSRF enabled in the default app configuration.
Authorization	RBAC across customer, teller, manager, and admin workflows.
Database safety	SQLAlchemy-backed repository access and parameterized query behavior.
XSS prevention	Jinja template auto-escaping and security tests for XSS payload escaping.
Replay prevention	Transaction nonces prevent duplicate replay submission.
Financial integrity	Ledger-backed balance calculation, insufficient-funds protection, and atomic transaction processing.
Fraud detection	Large transfer, new account transfer, and high-velocity transaction checks.
Audit integrity	HMAC-backed tamper-evident audit log hash chain.
Configuration hardening	Application rejects missing or placeholder production SECRET_KEY values.

6. Secure Testing Strategy

The project uses a mixed testing strategy. Integration tests exercise full Flask route behavior through the Flask test client. Security tests simulate adversarial web and banking cases. Unit tests validate the correctness of services, repositories, middleware, fraud rules, audit chain generation, and transaction service behavior.

Test Layer	Purpose	Examples
Integration	Validate end-to-end route behavior and authentication flow.	Login page, sign-in alias, signup, MFA page, logout, dashboard login requirements, account dashboard, deposit route, insufficient-funds withdrawal.
Security	Validate resistance to common web and banking attacks.	Brute-force lockout, rate limiting, unauthorized admin access rejection, replay rejection, SQL injection handling, XSS escaping.
Unit	Validate isolated logic and internal security controls.	Password hashing, OTP verification, account creation, balance calculation, audit hash-chain verification, fraud detection, RBAC, rate limiter, transaction service.

7. Security Test Analysis

Security Scenario	Technique	Expected Secure Behavior	Reported Result
Brute-force login	Repeated failed login attempts.	Account is locked after the configured failed-attempt threshold.	Covered by security tests.
Rate limit abuse	Repeated requests inside configured time window.	Requests beyond the limit are blocked/throttled.	Covered by security tests.
Unauthenticated dashboard access	Direct request to protected dashboard URL.	User is redirected or rejected until authenticated.	Covered by integration/security tests.
Customer attempts admin access	Authenticated customer requests admin route.	Access is denied because customer role lacks admin privileges.	Covered by security tests.
Audit visibility	Admin accesses audit log interface.	Authorized admin can view audit logs; non-authorized roles cannot.	Covered by security tests.
Replay attack	Duplicate transaction submission with reused nonce.	Repeated transaction is rejected.	Covered by security and unit tests.
SQL injection	Authentication/input payload contains SQL injection patterns.	Payload is handled as data, not executable SQL.	Covered by security tests.
XSS	Script payload submitted into rendered fields.	Payload is escaped in template output.	Covered by security tests.
Audit log tampering	Audit chain verification is run after log creation.	Chain verification detects inconsistency if records are changed.	Covered by unit tests.
Insufficient funds	Withdrawal/transfer exceeds calculated balance.	Transaction fails and balance is not corrupted.	Covered by integration/unit tests.

8. Secure Code Review

8.1 Authentication and Credential Handling

The code review confirms that plaintext password storage is avoided by using bcrypt hashing. Authentication is reinforced by failed-login tracking and lockout behavior.

8.2 Authorization and Access Control

The authorization model is appropriate for the project scope because it distinguishes customer, teller, manager, and admin responsibilities. The most important security property is that customers must not be able to access institution-wide account data, fraud review, audit logs, or user-management functions. Tests explicitly cover customer restriction on admin pages and admin access to audit logs.

8.3 Input Handling and Web Attack Resistance

The application relies on Flask-WTF for CSRF protection in the normal/default configuration. Jinja2 template escaping is used for frontend rendering, and security tests exercise XSS payload escaping. SQLAlchemy-backed data access reduces SQL injection risk by preventing string-concatenated SQL query execution patterns in normal repository usage.

8.4 Transaction Integrity

The banking domain makes transaction integrity more important than visual completeness. SCBS uses ledger-backed balance calculation instead of trusting a manually stored balance field. Deposit, withdrawal, and transfer behavior is tested, including insufficient-funds rejection and replay prevention through nonces.

8.5 Auditability and Tamper Evidence

Audit logging is treated as a security control, not a cosmetic feature. The system records login, transaction, account lock, and other security events. HMAC-backed hash chaining provides tamper evidence: changing prior audit records should invalidate later chain verification.

8.6 Configuration Review

The configuration rejects missing or placeholder production SECRET_KEY values at application startup, while the testing configuration uses a separate test secret and test database. Cookie settings include HTTP-only behavior and SameSite=Lax. Flask debug mode is not forced on by the application factory.

9. Final Security Fixes and Enhancements

Fix / Enhancement	Security Value
Placeholder SECRET_KEY rejection	Prevents the app from starting in non-test mode with known weak placeholder secrets.
CSRF-enabled default configuration	Protects state-changing form submissions against cross-site request forgery.
IP-bound session guard	Raises the cost of session hijacking by tying sessions to the client IP context.
Session timeout behavior	Limits exposure from abandoned or stale authenticated sessions.
Rate limiting and login lockout	Reduces brute-force and credential stuffing success probability.
Transaction nonce replay checks	Stops duplicate transaction submissions from being accepted.
Fraud detection workflow	Flags high-risk money movement for manager/admin review.
Tamper-evident audit logs	Improves accountability and makes silent audit manipulation detectable.
Frontend redesign with role-aware surfaces	Makes fraud, audit, admin, and transaction workflows visible and demo-ready.

10. Reported Test Results

The repository handoff states that the latest full test run completed with 59 passed tests. The report records this as the project-reported test result. It should be re-run immediately before final submission using the command below so the team can truthfully state the result in the presentation.

```
.\venv\Scripts\python -m pytest tests -v
```

Metric	Reported Status
Full test suite	59 passed
Integration coverage	Route/auth/account transaction behavior covered
Security coverage	Brute force, rate limiting, RBAC rejection, replay, SQLi, XSS covered
Unit coverage	Services, repositories, middleware, audit chain, fraud logic, transaction behavior covered
Known warnings	datetime.utcnow() deprecation and Flask-Session filesystem backend deprecation warnings remain